WebSocket Lab - Tasks

Duration: 2.5 hours | **Points:** 100

Lab Overview

Objective: Learn to build real-time WebSocket services in Go, covering connection management, JSON message protocols, broadcasting with the Hub pattern, rooms, private messaging, and production-quality connection lifecycle handling.

What You'll Build: A real-time Chat Application where users connect with usernames, join chat rooms, send messages that are broadcast to all room members, send private direct messages, and receive join/leave notifications — all over WebSocket using the Hub pattern.

Why This Matters for Your Project:
The skills you learn here apply directly to your MangaHub project:

- WebSocket upgrade + read loop → Real-time connection handling for live features
- JSON message protocol → Structured event messages (new chapter alerts, reading progress)
- Hub pattern + broadcasting → Notify all subscribers when new manga chapters are released
- Rooms → Genre-based or manga-specific notification channels
- Private messaging → Direct notifications to specific users (e.g., friend recommendations)
- Connection lifecycle → Handling readers joining/leaving, detecting dropped connections

Key Transferable Skills:

- ✅ WebSocket server with `gorilla/websocket`
- ✅ JSON-based message protocol design
- ✅ Hub pattern for multi-client coordination
- ✅ Read pump / write pump goroutine architecture
- ✅ Room-based targeted broadcasting
- ✅ Private messaging with username routing
- ✅ Connection lifecycle management (ping/pong, graceful close)
- ✅ Input validation and error handling

Setup (5 minutes)

Create New Project

```
mkdir live-chat
cd live-chat
go mod init live-chat
```

Install Required Packages

```
go get github.com/gorilla/websocket
```

Project Structure

```
live-chat/
├── go.mod
├── hub.go          ← you create this (Hub struct + run loop)
├── client.go       ← you create this (Client struct + read/write pumps)
├── main.go         ← you create this (HTTP server + WebSocket handler)
├── go-client/
│   └── main.go     ← you create this (Go terminal client)
└── static/
    └── index.html  ← you create this (browser client)
```

📚 Study Materials

Before starting the tasks, make sure to read through the **WebSocket Sample Code Guide** file which contains:

- WebSocket basics with the Upgrader and echo pattern
- JSON message protocol design
- Hub pattern for broadcasting
- Client struct with read/write pumps
- Go client for WebSocket
- Connection management and error handling
- Testing techniques
- Common mistakes and solutions

✍️ Task 1: WebSocket Echo Server with JSON Messages (15 points)

🎯 Real-World Application

Every WebSocket system starts with the basic connection lifecycle: upgrade, read, write, close. When you open WhatsApp Web, the browser first establishes a WebSocket connection and exchanges JSON-formatted messages with the server. In this task, you'll implement the foundational pattern — a server that accepts structured JSON messages, validates them, and responds — the same pattern used by virtually every real-time web application.

Challenge: Build a WebSocket server that accepts JSON messages, validates their structure, and echoes them back with a server-added timestamp.

Instructions:

1. **Define a `Message` struct** with fields: `Type` (string), `Sender` (string), `Content` (string), `Timestamp` (string, omitempty)

2. **Implement the WebSocket handler:**
    - Configure an `Upgrader` with `CheckOrigin` allowing all origins
    - Upgrade the HTTP connection to WebSocket
    - Run a read loop that:
        - Parses incoming messages as JSON
        - Validates that `type` and `content` are non-empty
        - Sends back an error message (type `"error"`) for invalid input
        - For valid `"chat"` messages, adds a timestamp and echoes back
        - Logs each received message

3. **Create a browser client** (`static/index.html`) that:
    - Connects to `ws://localhost:8080/ws`
    - Has input fields for name and message
    - Sends JSON messages with type `"chat"`
    - Displays received messages and errors

Starter Code (`main.go`):

```go
package main

import (
        "encoding/json"
        "log"
        "net/http"
        "time"

        "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
        ReadBufferSize:  1024,
        WriteBufferSize: 1024,
        // TODO: Configure CheckOrigin to allow all origins for development
}

// Message defines the JSON structure for WebSocket communication.
type Message struct {
        // TODO: Define fields — Type, Sender, Content, Timestamp
        // Hint: Use json struct tags. Timestamp should have omitempty.
}

func handleWebSocket(w http.ResponseWriter, r *http.Request) {
        // TODO: Upgrade HTTP connection to WebSocket
        // Hint: conn, err := upgrader.Upgrade(w, r, nil)

        // TODO: defer conn.Close()

        log.Printf("client connected: %s", conn.RemoteAddr())

        // TODO: Read loop
        for {
                // TODO: Read message with conn.ReadMessage()

                // TODO: Parse JSON with json.Unmarshal into Message struct
                // Hint: If parsing fails, send an error message back and continue

                // TODO: Validate that Type and Content are non-empty
                // Hint: If validation fails, send an error message back and continu

                // TODO: Log the received message

                // TODO: Route based on msg.Type using a switch statement
                // For "chat": add timestamp (time.Now().Format(time.RFC3339)) and e
                // For unknown types: send an error message

                // TODO: Marshal the response and send with conn.WriteMessage
        }
}

func main() {
        http.HandleFunc("/ws", handleWebSocket)
        http.Handle("/", http.FileServer(http.Dir("static")))

        log.Println("Echo server listening on :8080")
        log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Starter Code (`static/index.html`):

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WebSocket Echo</title>
    <style>
        body { font-family: monospace; max-width: 600px; margin: 40px auto; }
        #log { border: 1px solid #ccc; padding: 10px; height: 300px; overflow-y: aut
               background: #f9f9f9; margin-bottom: 10px; }
        .sent { color: blue; }
        .received { color: green; }
        .error { color: red; }
        .system { color: gray; font-style: italic; }
        input { padding: 5px; }
        button { padding: 5px 15px; }
    </style>
</head>
<body>
    <h2>WebSocket Echo</h2>
    <div id="log"></div>
    <input type="text" id="name" placeholder="Your name" value="Alice" style="width:
    <input type="text" id="msg" placeholder="Type a message..." style="width:300px"
    <button onclick="send()">Send</button>

    <script>
        const logDiv = document.getElementById('log');
        const nameInput = document.getElementById('name');
        const msgInput = document.getElementById('msg');

        function appendLog(text, cls) {
            const d = document.createElement('div');
            d.textContent = text;
            d.className = cls;
            logDiv.appendChild(d);
            logDiv.scrollTop = logDiv.scrollHeight;
        }

        // TODO: Create WebSocket connection to ws://localhost:8080/ws

        // TODO: Set up ws.onopen — log "Connected"
        // TODO: Set up ws.onmessage — parse JSON, display based on msg.type
        // TODO: Set up ws.onclose — log "Disconnected"

        function send() {
            const text = msgInput.value;
            if (!text || ws.readyState !== WebSocket.OPEN) return;

            // TODO: Build a JSON message with type "chat", sender from nameInput, c
            // TODO: Send with ws.send(JSON.stringify(msg))
            // TODO: Display the sent message in the log
            // TODO: Clear the input field
        }

        msgInput.addEventListener('keypress', (e) => { if (e.key === 'Enter') send()
    </script>
</body>
</html>
```

Run and test:

```
go run .
# Open http://localhost:8080 in your browser
```

Expected behavior:

- Send `{"type":"chat","sender":"Alice","content":"Hello"}` → receive same message back with timestamp added
- Send `{"type":"unknown","sender":"Alice","content":"test"}` → receive `{"type":"error","content":"unknown message type: unknown"}`
- Send `invalid json` → receive `{"type":"error","content":"invalid JSON format"}`

Grading Criteria (15 points):

- ✅ Message struct defined with correct JSON tags (2 points)
- ✅ Upgrader configured and connection upgrades successfully (2 points)
- ✅ Read loop processes messages correctly (2 points)
- ✅ JSON parsing with error handling (invalid JSON sends error back) (2 points)
- ✅ Validation rejects empty `type` and `content` fields (2 points)
- ✅ Chat messages echoed back with server timestamp (2 points)
- ✅ Browser client sends and receives JSON messages (2 points)
- ✅ Server logs each received message (1 point)

✍️ Task 2: Hub Pattern — Broadcast Chat (20 points)

🎯 Real-World Application

When Discord broadcasts a message to everyone in a server channel, or when a live sports app pushes score updates to all viewers, they're using the broadcast pattern. A central coordinator (the Hub) receives messages from any source and distributes them to all connected clients. This is the foundation of every multi-user real-time system — and the exact pattern you'll use in MangaHub to notify all subscribers when a new manga chapter is released.

Challenge: Implement the Hub pattern to broadcast messages from any connected client to all clients, with join/leave notifications.

Instructions:

1. **Create `hub.go`** — the Hub struct with:
    - `clients` map, `broadcast` / `register` / `unregister` channels
    - `run()` method with select loop
    - When broadcasting, handle full send buffers with a `default` case

2. **Create `client.go`** — the Client struct with:
    - `hub`, `conn`, `send` channel, `username` string
    - `readPump()` — reads from WebSocket, parses JSON, forwards to `hub.broadcast`
    - `writePump()` — drains `send` channel, writes to WebSocket, sends periodic pings
    - Ping/pong keep-alive with deadlines

3. **Update `main.go`** — create Hub, launch `hub.run()`, wire up `serveWs` handler

4. **Add join/leave notifications** — when a client connects, broadcast a `"join"` message; when they disconnect, broadcast a `"leave"` message

5. **Create a Go client** (`go-client/main.go`) — connects to the server, reads/sends messages from terminal

Starter Code (`hub.go`):

```go
package main

import (
        "encoding/json"
        "log"
)

type Hub struct {
        clients    map[*Client]bool
        broadcast  chan []byte
        register   chan *Client
        unregister chan *Client
}

func newHub() *Hub {
        return &Hub{
                // TODO: Initialize all fields
        }
}

func (h *Hub) run() {
        for {
                select {
                case client := <-h.register:
                        // TODO: Add client to the map
                        // TODO: Log how many clients are connected
                        // TODO: Broadcast a "join" message to all clients
                        // Hint: Create a Message{Type: "join", Sender: client.userna
                        // Hint: Marshal to JSON and send to h.broadcast

                case client := <-h.unregister:
                        // TODO: Check if client exists in the map
                        // TODO: Delete from map and close their send channel
                        // TODO: Broadcast a "leave" message to all clients

                case message := <-h.broadcast:
                        // TODO: Send the message to every connected client
                        // TODO: Use a non-blocking send (select with default case)
                        // TODO: If a client's send buffer is full, disconnect them
                }
        }
}
```

Starter Code (`client.go`):

```go
package main

import (
        "encoding/json"
        "log"
        "time"

        "github.com/gorilla/websocket"
)

const (
        writeWait      = 10 * time.Second
        pongWait       = 60 * time.Second
        pingPeriod     = (pongWait * 9) / 10
        maxMessageSize = 512
)

type Client struct {
        hub      *Hub
        conn     *websocket.Conn
        send     chan []byte
        username string
}

func (c *Client) readPump() {
        defer func() {
                // TODO: Unregister from hub
                // TODO: Close the connection
        }()

        c.conn.SetReadLimit(maxMessageSize)
        c.conn.SetReadDeadline(time.Now().Add(pongWait))
        c.conn.SetPongHandler(func(string) error {
                c.conn.SetReadDeadline(time.Now().Add(pongWait))
                return nil
        })

        for {
                _, rawMessage, err := c.conn.ReadMessage()
                if err != nil {
                        // TODO: Log unexpected close errors
                        // Hint: Use websocket.IsUnexpectedCloseError
                        break
                }

                // TODO: Parse JSON message
                // TODO: Validate required fields (type, content)
                // TODO: Override sender with c.username (prevent spoofing)
                // TODO: Add timestamp
                // TODO: Marshal and send to hub.broadcast

                log.Printf("[%s] %s: %s", msg.Type, c.username, msg.Content)
        }
}

func (c *Client) writePump() {
        ticker := time.NewTicker(pingPeriod)
        defer func() {
                ticker.Stop()
                c.conn.Close()
        }()

        for {
                select {
                case message, ok := <-c.send:
                        // TODO: Set write deadline
                        // TODO: If channel closed (!ok), send close frame and retur
                        // TODO: Write the message to the WebSocket

                case <-ticker.C:
                        // TODO: Set write deadline
                        // TODO: Send ping message
                        // TODO: If ping fails, return (connection is dead)
                }
        }
}
```

Starter Code (`main.go`):

```go
package main

import (
        "log"
        "net/http"

        "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
        ReadBufferSize:  1024,
        WriteBufferSize: 1024,
        CheckOrigin: func(r *http.Request) bool { return true },
}

func serveWs(hub *Hub, w http.ResponseWriter, r *http.Request) {
        conn, err := upgrader.Upgrade(w, r, nil)
        if err != nil {
                log.Println("upgrade error:", err)
                return
        }

        // TODO: Get username from query parameter: r.URL.Query().Get("username")
        // TODO: If empty, default to "Anonymous"

        // TODO: Create Client struct with hub, conn, send channel (buffer 256), use

        // TODO: Register client with hub

        // TODO: Launch writePump and readPump as goroutines
}

func main() {
        hub := newHub()
        go hub.run()

        http.HandleFunc("/ws", func(w http.ResponseWriter, r *http.Request) {
                serveWs(hub, w, r)
        })
        http.Handle("/", http.FileServer(http.Dir("static")))

        log.Println("Chat server listening on :8080")
        log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Update `static/index.html` to:

- Prompt for a username before connecting
- Connect with username as query param: `ws://localhost:8080/ws?username=Alice`
- Display join/leave/chat messages differently

Starter Code (`go-client/main.go`):

```go
package main

import (
        "bufio"
        "encoding/json"
        "fmt"
        "log"
        "os"
        "os/signal"

        "github.com/gorilla/websocket"
)

type Message struct {
        Type      string `json:"type"`
        Sender    string `json:"sender"`
        Content   string `json:"content"`
        Timestamp string `json:"timestamp,omitempty"`
}

func main() {
        if len(os.Args) < 2 {
                log.Fatal("usage: go run . <username>")
        }
        username := os.Args[1]

        // TODO: Connect to ws://localhost:8080/ws?username=<username>
        // Hint: url := fmt.Sprintf("ws://localhost:8080/ws?username=%s", username)
        // Hint: conn, _, err := websocket.DefaultDialer.Dial(url, nil)

        // TODO: Handle Ctrl+C with os.Signal

        // TODO: Goroutine to read and display server messages
        // Hint: Parse JSON, format output based on msg.Type

        // TODO: Main loop to read stdin and send chat messages
        // Hint: Build Message{Type: "chat", Sender: username, Content: text}
        // Hint: Send with conn.WriteMessage

        // TODO: On Ctrl+C, send CloseMessage with CloseNormalClosure
}
```

Run and test:

```
# Terminal 1 — server
go run .

# Terminal 2 — Go client
cd go-client && go run . Alice

# Terminal 3 — Go client
cd go-client && go run . Bob

# Also open http://localhost:8080 in browser
```

Expected behavior:

- When Alice connects: all clients see `"Alice joined the chat"`
- When Alice sends "Hello": all clients (including Alice) see the message
- When Bob disconnects: all clients see `"Bob left the chat"`

Grading Criteria (20 points):

- ✅ Hub struct with register/unregister/broadcast channels (3 points)
- ✅ Hub `run()` correctly adds/removes clients (2 points)
- ✅ Hub broadcasts messages to all connected clients (3 points)
- ✅ Hub handles full send buffers (disconnects stuck clients) (2 points)
- ✅ Client readPump reads, parses JSON, and forwards to Hub (2 points)
- ✅ Client writePump drains send channel and writes to WebSocket (2 points)
- ✅ Ping/pong keep-alive implemented (2 points)
- ✅ Join/leave notifications broadcast to all clients (2 points)
- ✅ Go client connects and sends/receives messages (2 points)

✍️ Task 3: Rooms — Targeted Broadcasting (20 points)

🎯 Real-World Application

Discord has servers with channels. Slack has workspaces with channels. Twitch has streams with chat rooms. All of these use the same pattern: group clients into rooms and only broadcast messages within the same room. In your MangaHub project, rooms could represent manga-specific notification channels — users subscribe to "One Piece" or "Naruto" and only receive updates for manga they follow.

Challenge: Extend the Hub to support named rooms. Clients join a room on connection and only receive messages from that room. Clients can switch rooms by sending a `"join_room"` message.

Instructions:

1. **Update the Hub** to track rooms:
    - Replace `clients map[*Client]bool` with `rooms map[string]map[*Client]bool`
    - Update register to add client to their room
    - Update unregister to remove from their room
    - Update broadcast to only send to clients in the same room

2. **Update the Client** to have a `room` field:
    - Default room is `"general"` if not specified
    - Accept room as query parameter: `ws://localhost:8080/ws?username=Alice&room=golang`

3. **Implement `join_room` message type:**
    - Client sends `{"type":"join_room","content":"new-room-name"}`
    - Server moves the client from their current room to the new room
    - Broadcast leave notification in old room, join notification in new room

4. **Implement `list_rooms` message type:**
    - Client sends `{"type":"list_rooms"}`
    - Server responds with a message containing all active room names and user counts

5. **Update the browser client** to show room controls (join room, current room indicator)

Starter Code (Hub updates):

```go
type Hub struct {
        rooms      map[string]map[*Client]bool // room name → set of clients
        broadcast  chan *RoomMessage            // messages tagged with room
        register   chan *Client
        unregister chan *Client
}

// RoomMessage wraps a message with the target room.
type RoomMessage struct {
        room    string
        message []byte
}

func newHub() *Hub {
        return &Hub{
                rooms:      make(map[string]map[*Client]bool),
                broadcast:  make(chan *RoomMessage),
                register:   make(chan *Client),
                unregister: make(chan *Client),
        }
}

func (h *Hub) run() {
        for {
                select {
                case client := <-h.register:
                        // TODO: Create the room if it doesn't exist
                        // TODO: Add client to their room
                        // TODO: Broadcast join notification to that room only
                        // TODO: Log room membership count

                case client := <-h.unregister:
                        // TODO: Find which room the client is in
                        // TODO: Remove client from that room
                        // TODO: If room is empty, delete it from the map
                        // TODO: Close client's send channel
                        // TODO: Broadcast leave notification to that room

                case roomMsg := <-h.broadcast:
                        // TODO: Get the set of clients for roomMsg.room
                        // TODO: Send roomMsg.message to each client in that room onl
                        // TODO: Handle full send buffers (disconnect stuck clients)
                }
        }
}

// moveClientToRoom moves a client from their current room to a new room.
func (h *Hub) moveClientToRoom(client *Client, newRoom string) {
        // TODO: Remove client from current room
        // TODO: Broadcast leave notification in old room
        // TODO: Update client.room
        // TODO: Add client to new room (create if needed)
        // TODO: Broadcast join notification in new room
}
```

Starter Code (readPump update for room routing):

```
// Inside readPump, after parsing the message:
switch msg.Type {
case "chat":
        // TODO: Add timestamp and sender
        // TODO: Marshal and send as RoomMessage to hub.broadcast
        // Hint: c.hub.broadcast <- &RoomMessage{room: c.room, message: outBytes}

case "join_room":
        // TODO: Validate new room name is non-empty
        // TODO: Call hub.moveClientToRoom(c, msg.Content)
        // TODO: Send confirmation message back to the client

case "list_rooms":
        // TODO: Build a response listing all rooms and their user counts
        // TODO: Send response directly to this client (not broadcast)

default:
        // TODO: Send error message for unknown types
}
```

Testing:

```
# Terminal 1 — server
go run .

# Terminal 2 — Alice joins "golang" room
cd go-client && go run . Alice golang

# Terminal 3 — Bob joins "golang" room
cd go-client && go run . Bob golang

# Terminal 4 — Charlie joins "python" room
cd go-client && go run . Charlie python
```

Expected behavior:

- Alice and Bob see each other's messages (same room)
- Charlie does NOT see Alice's or Bob's messages (different room)
- If Bob sends `{"type":"join_room","content":"python"}`:
  - Alice sees "Bob left the room"
  - Charlie sees "Bob joined the room"
  - Bob now sees Charlie's messages, not Alice's

Grading Criteria (20 points):

- ✅ Hub tracks rooms with `map[string]map[*Client]bool` (3 points)
- ✅ Clients join correct room on connection (2 points)
- ✅ Messages broadcast only to clients in the same room (4 points)
- ✅ `join_room` moves client between rooms correctly (3 points)
- ✅ Join/leave notifications sent to the correct room (2 points)
- ✅ Empty rooms are cleaned up from the map (2 points)
- ✅ `list_rooms` returns accurate room names and user counts (2 points)
- ✅ Go client accepts room as command-line argument (2 points)

✍️ Task 4: Private Messaging — Direct Client-to-Client (20 points)

🎯 Real-World Application

When you send a direct message on Slack or Instagram, the server routes your message to one specific user — not to everyone. This requires the server to maintain a mapping of usernames to connections, validate recipients, and handle cases where the target user is offline. In MangaHub, private messaging could power friend recommendations ("Hey, you should read this manga!") or admin notifications to specific users.

Challenge: Implement direct messaging where a client can send a message to a specific user by username. The server routes the message only to the target client.

Instructions:

1. **Update the Hub** to track usernames:
   - Add `usernames map[string]*Client` — maps username → Client
   - On register, add to the username map
   - On unregister, remove from the username map
   - Reject duplicate usernames (send error to client)
2. **Implement `dm` message type:**
   - Client sends `{"type":"dm","content":"Hey!","recipient":"Bob"}`
   - Server finds Bob's Client by username
   - Server sends the message only to Bob (and a confirmation to the sender)
   - If Bob is not online, send an error back to the sender
3. **Add `Recipient` field to Message struct:**
   - `Recipient string \` json:"recipient,omitempty"``
4. **Update the browser client** to support DMs (e.g., `/dm Bob Hello!` syntax)

Starter Code (Hub additions):

```
type Hub struct {
        rooms      map[string]map[*Client]bool
        usernames  map[string]*Client           // username → client
        broadcast  chan *RoomMessage
        register   chan *Client
        unregister chan *Client
        directMsg  chan *DirectMessage           // private messages
}

// DirectMessage carries a message from one user to another.
type DirectMessage struct {
        sender    *Client
        recipient string
        message   []byte
}

func newHub() *Hub {
        return &Hub{
                rooms:      make(map[string]map[*Client]bool),
                usernames:  make(map[string]*Client),
                broadcast:  make(chan *RoomMessage),
                register:   make(chan *Client),
                unregister: make(chan *Client),
                directMsg:  make(chan *DirectMessage),
        }
}

func (h *Hub) run() {
        for {
                select {
                case client := <-h.register:
                        // TODO: Check if username is already taken
                        // If taken: send error message to client, close connection
                        // If not: add to usernames map and room map
                        // Broadcast join notification

                case client := <-h.unregister:
                        // TODO: Remove from usernames map
                        // TODO: Remove from room map
                        // TODO: Broadcast leave notification

                case roomMsg := <-h.broadcast:
                        // ... (same as Task 3)

                case dm := <-h.directMsg:
                        // TODO: Look up recipient in h.usernames
                        // TODO: If found, send the message to recipient's send chann
                        // TODO: Also send a confirmation to the sender
                        // TODO: If not found, send an error to the sender
                }
        }
}
```

Starter Code (Message struct update):

```
type Message struct {
        Type      string `json:"type"`
        Sender    string `json:"sender"`
        Content   string `json:"content"`
        Recipient string `json:"recipient,omitempty"`
        Timestamp string `json:"timestamp,omitempty"`
}
```

Starter Code (readPump addition):

```
case "dm":
        // TODO: Validate that msg.Content is non-empty
        // TODO: Validate that msg.Recipient is non-empty
        // TODO: Prevent sending DM to yourself
        // TODO: Build outgoing message with sender, timestamp, type "dm"
        // TODO: Marshal and send via hub.directMsg channel
        // Hint: c.hub.directMsg <- &DirectMessage{
        //     sender: c, recipient: msg.Recipient, message: outBytes,
        // }
```

Testing:

```
# Terminal 1 — server
go run .

# Terminal 2 — Alice
cd go-client && go run . Alice general

# Terminal 3 — Bob
cd go-client && go run . Bob general

# Terminal 4 — Charlie
cd go-client && go run . Charlie general
```

Expected behavior:

- Alice sends `{"type":"dm","recipient":"Bob","content":"Secret message"}`:
  - Bob receives the message
  - Alice receives a confirmation (e.g., `"DM sent to Bob"`)
  - Charlie does NOT receive anything
- Alice sends `{"type":"dm","recipient":"Dave","content":"Hello"}`:
  - Alice receives an error: `"user not found: Dave"`
- Alice tries to use username "Bob" (already taken):
  - Connection is rejected with error: `"username already taken: Bob"`

Grading Criteria (20 points):

- ✅ Hub maintains `usernames` map (2 points)
- ✅ Duplicate usernames rejected with error message (3 points)
- ✅ DM routed only to the target recipient (4 points)
- ✅ Sender receives confirmation of delivered DM (2 points)
- ✅ Error returned for non-existent recipient (2 points)
- ✅ Cannot DM yourself (1 point)
- ✅ Usernames cleaned up on disconnect (2 points)
- ✅ Go client supports sending DMs (2 points)
- ✅ Regular broadcast still works alongside DMs (2 points)

✍️ Task 5: Complete Service with Connection Lifecycle & Error Handling (25 points) — REQUIRED

🎯 Real-World Application

Production WebSocket services at Slack and Discord handle millions of concurrent connections. They validate all input, reject malformed messages, enforce rate limits, handle graceful disconnection, and provide tools for users to query the system state. This task brings your chat server to production quality — the same patterns you'll need when MangaHub handles real users with real network conditions.

Challenge: Add production-quality features to the complete system built in Tasks 1–4.

Instructions:

1. **Username validation:**
   - Non-empty, alphanumeric + underscores only, 3–20 characters
   - Reject with descriptive error message
2. **Message validation:**
   - Content must be non-empty and ≤ 500 characters
   - Type must be one of: `chat`, `dm`, `join_room`, `list_rooms`, `list_users`
   - Reject with descriptive error message
3. **Implement `list_users` message type:**
   - Client sends `{"type":"list_users"}`
   - Server responds with all online users in the sender's current room
   - Response includes username and room name for each user
4. **Graceful disconnect handling:**
   - When a client sends a proper Close frame, remove username, broadcast a leave notification
   - When a client drops unexpectedly, still broadcast a leave notification
   - Clean up username and room membership in both cases
5. **Request logging:**
   - Log every message with: timestamp, username, message type, room, content summary
   - Log every connect/disconnect with: timestamp, username, room, total connected
6. **Error handling improvements:**
   - All error responses use `{"type":"error","content":"descriptive message"}`
   - Server never panics on bad input
   - Malformed JSON is handled gracefully (error sent, connection stays open)

Starter Code (validation helpers):

```
import "regexp"

var usernameRegex = regexp.MustCompile(`^[a-zA-Z0-9_]{3,20}$`)

func validateUsername(username string) error {
        // TODO: Check non-empty
        // TODO: Check matches regex pattern
        // TODO: Return descriptive error if invalid
        return nil
}

func validateMessage(msg *Message) error {
        // TODO: Check Content is non-empty
        // TODO: Check Content length ≤ 500
        // TODO: Check Type is one of the allowed types
        // TODO: For "dm" type, check Recipient is non-empty
        // TODO: Return descriptive error if invalid
        return nil
}
```

Starter Code (`list_users` in readPump):

```
case "list_users":
        // TODO: Get all clients in the sender's current room from the Hub
        // TODO: Build a list of usernames
        // TODO: Send response with type "user_list" and content as comma-separated
        // Hint: You may need to add a method to Hub or access the rooms map
        // Note: Be careful about concurrency — the Hub runs in its own goroutine
        // Option A: Add a list_users channel to Hub (safest)
        // Option B: Use sync.RWMutex to protect the rooms map (simpler)
```

Testing:

```
# Test username validation
cd go-client && go run . "" general
# Expected: connection rejected, error "username is required"

cd go-client && go run . "ab" general
# Expected: connection rejected, error "username must be 3–20 alphanumeric character

cd go-client && go run . "Alice!" general
# Expected: connection rejected, error "username must be 3–20 alphanumeric character

# Test message validation
# Send {"type":"chat","content":""} → error "content is required"
# Send {"type":"invalid","content":"test"} → error "unknown message type: invalid"

# Test list_users
# Connect Alice and Bob to "general", send {"type":"list_users"}
# Expected: {"type":"user_list","content":"Alice, Bob"}

# Test graceful disconnect
# Connect Alice, Bob, Charlie
# Alice Ctrl+C → Bob and Charlie see "Alice left the chat"
# Verify no panic on server, connection cleanup is clean
```

Expected server logs:

```
2024/01/15 14:30:01 [CONNECT] Alice joined room "general" (3 online)
2024/01/15 14:30:05 [CHAT] Alice@general: Hello everyone
2024/01/15 14:30:08 [DM] Alice → Bob: Secret message
2024/01/15 14:30:12 [JOIN_ROOM] Alice moved from "general" to "golang"
2024/01/15 14:30:15 [DISCONNECT] Alice left room "golang" (2 online)
```

Grading Criteria (25 points):

- ✅ Username validation: non-empty, 3–20 chars, alphanumeric + underscore (3 points)
- ✅ Username validation: rejects invalid with descriptive error (2 points)
- ✅ Message validation: content non-empty, ≤ 500 chars (2 points)
- ✅ Message validation: type must be a recognized value (2 points)
- ✅ `list_users` returns correct users in the current room (3 points)
- ✅ Graceful disconnect broadcasts leave notification in both clean and abrupt cases (3 points)
- ✅ Username and room cleaned up on disconnect (2 points)
- ✅ Request logging with timestamp, username, type, room (3 points)
- ✅ All errors use consistent JSON format with descriptive messages (2 points)
- ✅ Server never panics on bad input (malformed JSON, empty messages, etc.) (2 points)
- ✅ Code quality — clean, well-organized, no race conditions (1 point)

Submission Requirements (100 points)

Tasks (85 points)

- ☐ Task 1: WebSocket Echo Server with JSON Messages (15 points)
- ☐ Task 2: Hub Pattern — Broadcast Chat (20 points)
- ☐ Task 3: Rooms — Targeted Broadcasting (20 points)
- ☐ Task 4: Private Messaging — Direct Client-to-Client (20 points)
- ☐ Task 5: Complete Service with Connection Lifecycle & Error Handling (25 points) — **REQUIRED**

Code Quality (15 points)

- ☐ Server compiles and runs without panics (3 points)
- ☐ Browser client connects and demonstrates all features (3 points)
- ☐ Go client compiles and demonstrates chat, DM, and room switching (3 points)
- ☐ JSON message protocol is consistent across all features (3 points)
- ☐ Clean, readable code with comments explaining key decisions (3 points)

Bonus (+15 points)

- ☐ Add a typing indicator — client sends `{"type":"typing"}`, server broadcasts `"Alice is typing..."` to the room, auto-clears after 3 seconds (+3 points)
- ☐ Add message history — store the last 50 messages per room in memory, send them to new clients when they join a room (+4 points)
- ☐ Containerize with Docker — `Dockerfile` for the server and `docker-compose.yml` with a Go client connecting by service name (+5 points)
- ☐ Add reconnection with exponential backoff in the JavaScript client — auto-reconnect on disconnect with 1s → 2s → 4s → ... → 30s delay, reset on successful connect (+3 points)

Testing Checklist

Before submitting, verify:

1. ✓ Server starts without errors on `:8080`
2. ✓ Browser client connects and sends/receives JSON messages
3. ✓ Go client connects with username and room arguments
4. ✓ Invalid JSON sends an error back (connection stays open)
5. ✓ Empty content rejected with descriptive error
6. ✓ Messages broadcast to all clients in the same room
7. ✓ Messages do NOT leak to clients in different rooms
8. ✓ Join/leave notifications appear in the correct room
9. ✓ `join_room` moves a client and notifies both rooms
10. ✓ `list_rooms` shows active rooms with user counts
11. ✓ DM reaches only the target recipient
12. ✓ DM to non-existent user returns an error
13. ✓ Duplicate username is rejected with an error
14. ✓ `list_users` shows users in the current room
15. ✓ Graceful disconnect triggers leave notification
16. ✓ Abrupt disconnect (kill process) still triggers cleanup
17. ✓ No deadlocks or panics under any input
18. ✓ Server logs all events with timestamps

Connection to Final Project (MangaHub)

This lab directly prepares you for the MangaHub project by teaching transferable WebSocket patterns:

What You Practiced (Chat) → What You'll Build (MangaHub):

1. **Task 1**: JSON echo → **Apply to**: Structured event messages for manga updates
2. **Task 2**: Broadcast chat → **Apply to**: Notify all subscribers of new chapter releases
3. **Task 3**: Room-based broadcast → **Apply to**: Manga-specific or genre-specific notification channels
4. **Task 4**: Private messaging → **Apply to**: Direct notifications (friend recommendations, admin alerts)
5. **Task 5**: Production lifecycle → **Apply to**: Robust handling of real users with real network conditions

Direct Skill Transfer:

Lab Pattern (Chat)	Project Application (MangaHub)
`Message{Type: "chat"}`	`Message{Type: "new_chapter"}`
Hub broadcast to room	Notify subscribers of manga updates
Room "general" / "golang"	Room "one-piece" / "naruto"
`join_room` message	`subscribe` to manga notifications
DM to specific user	Direct notification to a reader
`list_users` in room	`list_subscribers` for a manga
Username validation	User authentication check
Ping/pong keep-alive	Detect disconnected readers
Reconnection with backoff	Mobile app reconnection on network change

Code Patterns You Can Reuse:

```
// Lab Pattern (Chat)
type Message struct {
    Type    string `json:"type"`
    Sender  string `json:"sender"`
    Content string `json:"content"`
}

// Project Pattern (MangaHub) — Same structure!
type Notification struct {
    Type    string `json:"type"`    // "new_chapter", "update", "recommendation"
    Manga   string `json:"manga"`
    Content string `json:"content"` // "Chapter 1092 released!"
}

// Lab Pattern (Room broadcast)
hub.broadcast <- &RoomMessage{room: "general", message: data}

// Project Pattern (Same!)
hub.broadcast <- &RoomMessage{room: "one-piece", message: data}

// Lab Pattern (DM)
hub.directMsg <- &DirectMessage{sender: c, recipient: "Bob", message: data}

// Project Pattern (Same!)
hub.directMsg <- &DirectMessage{sender: system, recipient: "reader123", message: dat
```

What Changes for MangaHub:

1. **Entity names**: username → reader, room → manga/genre channel
2. **Message types**: `chat` → `new_chapter`, `join_room` → `subscribe`
3. **Data store**: in-memory → MongoDB (from your NoSQL lab)
4. **Transport**: `localhost` → Docker service names
5. **Authentication**: query parameter → JWT token verification
6. **Everything else**: Same Hub pattern, same readPump/writePump, same JSON protocol!

Getting Help

During Lab:

- 🤖 Ask AI: "How do I broadcast to a specific room in Go WebSocket?" or "gorilla/websocket ping pong example"
- 👥 Ask classmates
- 👨‍🏫 Ask instructor
- 📚 Check the Sample Code Guide (WebSocket_Lab_Sample_Code_Guide.md)

Useful Prompts for AI:

- "Go gorilla/websocket Hub pattern example"
- "How to implement rooms in a WebSocket chat server"
- "Go WebSocket private messaging by username"
- "gorilla/websocket ping pong keep-alive Go"
- "How to validate JSON messages in Go WebSocket server"
- "WebSocket reconnection with exponential backoff JavaScript"

Resources:

- gorilla/websocket Docs: https://pkg.go.dev/github.com/gorilla/websocket
- gorilla/websocket Chat Example: https://github.com/gorilla/websocket/tree/main/examples/chat
- MDN WebSocket API: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
- WebSocket RFC 6455: https://tools.ietf.org/html/rfc6455
- websocat (CLI tool): https://github.com/vi/websocat

End of WebSocket Lab Tasks